

BÀI THỰC HÀNH

LAB 17

SD Card Data Logger

Lưu dữ liệu ADC vào SD Card với EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	Data logging và lưu trữ dữ liệu	2
2.2	SD Card trong hệ thống nhúng	3
2.3	Filesystem và vòng đời của file	3
3	Thiết lập phần cứng	3
3.1	Phần cứng sử dụng	3
3.2	Ảnh xạ chân sử dụng	4
3.3	Kết nối mạch	4
4	EduFramework API	5
4.1	<code>SD_File_t</code>	5
4.2	<code>SD_Begin()</code>	5
4.3	<code>SD_Open()</code> và <code>SD_FILE_WRITE</code>	6
4.4	<code>SD_WriteLine()</code>	6
4.5	<code>SD_Close()</code> và <code>SD_End()</code>	7
5	Bài thực hành	7
5.1	Yêu cầu	7
5.2	Chương trình	7
5.3	Kiểm chứng	9
6	Mở rộng	9
6.1	Bài tập mở rộng	10
7	Tài liệu tham khảo	10

1 Giới thiệu

1.1 Tổng quan bài lab

Trong các hệ thống đo lường và giám sát, dữ liệu không phải lúc nào cũng chỉ được quan sát tại thời điểm chương trình đang chạy. Một data logger thu dữ liệu từ cảm biến hoặc tín hiệu đầu vào, xử lý bằng vi điều khiển và lưu lại để có thể truy xuất hoặc phân tích sau đó. Các hệ thống data logger sử dụng vi điều khiển và thẻ nhớ microSD đã được ứng dụng trong nhiều bài toán thu thập tín hiệu analog và giám sát dài hạn [1] [2]. Bài lab này sử dụng biến trở làm nguồn dữ liệu analog. S32K144 đọc giá trị ADC, sau đó lưu mười mẫu dữ liệu vào một file trên SD Card. Serial Monitor được dùng để theo dõi quá trình thu mẫu, còn dữ liệu lưu trên thẻ có thể được kiểm tra lại sau khi quá trình ghi hoàn tất.

Bài thực hành tập trung vào thao tác lưu trữ ở cấp ứng dụng. Các chi tiết giao tiếp SPI và xử lý giao thức SD ở mức thấp không được triển khai trong chương trình chính; EduFramework cung cấp SD Card Device API để thực hiện các thao tác khởi tạo, mở file, ghi dữ liệu và đóng tài nguyên.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả luồng dữ liệu cơ bản của một hệ thống data logging sử dụng vi điều khiển và bộ nhớ ngoài.
2. Giải thích vai trò của SD Card và filesystem trong việc lưu trữ dữ liệu của ứng dụng nhúng.
3. Kết nối module MicroSD và biến trở với MaaZEDU Development Board theo cấu hình của bài thực hành.
4. Sử dụng `SD_Begin()`, `SD_Open()`, `SD_WriteLine()`, `SD_Close()` và `SD_End()` để tạo và ghi dữ liệu vào file.
5. Kiểm chứng dữ liệu ADC đã được lưu trên SD Card sau khi chương trình kết thúc quá trình ghi.

2 Kiến thức nền

2.1 Data logging và lưu trữ dữ liệu

Data logging là quá trình thu thập dữ liệu theo thời gian và lưu lại để phục vụ việc quan sát hoặc phân tích sau đó. Một hệ thống có thể nhận dữ liệu từ cảm biến, tín hiệu analog hoặc các thiết bị đo, sau đó sử dụng vi điều khiển làm khối thu thập và xử lý trước khi ghi dữ liệu xuống bộ nhớ [1].

Mô hình tổng quát có thể biểu diễn như sau:

Nguồn dữ liệu → Thu thập → Vi điều khiển → Lưu trữ.

Trong bài lab này, biến trở tạo điện áp analog và `analogRead()` chuyển tín hiệu đó thành giá trị ADC thô. Giá trị được lưu vào file trên SD Card thay vì chỉ hiển thị trên Serial Monitor:

Potentiometer → ADC0_SE12 → S32K144 → SD Card → File.

Cách tổ chức này minh họa sự khác biệt giữa quan sát dữ liệu tức thời và lưu dữ liệu để sử dụng về sau. Các data logger dùng bộ nhớ tháo rời như microSD đặc biệt phù hợp khi hệ thống cần giữ lại nhiều lần đo mà không phụ thuộc vào kết nối liên tục với máy tính [2].

2.2 SD Card trong hệ thống nhúng

SD Card là thiết bị lưu trữ sử dụng bộ nhớ flash và có thể được sử dụng bởi host thông qua các chế độ giao tiếp được đặc tả bởi SD Association [3]. Trong EduFramework, SD Card Device sử dụng giao diện SPI đã có của framework, còn application chỉ làm việc với các API lưu trữ ở mức cao hơn [6].

Bài lab không yêu cầu chương trình ứng dụng tự gửi command của SD Card hoặc quản lý các block vật lý. Những thao tác này được xử lý bên dưới SD Device. Ở cấp ứng dụng, mục tiêu chỉ là khởi tạo thiết bị lưu trữ, làm việc với file và kết thúc phiên sử dụng đúng trình tự.

Ghi chú

SPI đã được sử dụng trong các bài lab trước. Lab 17 không lặp lại nguyên lý hoạt động của SPI mà chỉ sử dụng các đường SCK, MOSI, MISO và một chân chip-select để kết nối module MicroSD.

2.3 Filesystem và vòng đời của file

Thiết bị lưu trữ cung cấp không gian dữ liệu vật lý, trong khi filesystem tổ chức không gian đó thành các đối tượng mà ứng dụng có thể thao tác như file và thư mục. FatFs cung cấp các chức năng như mount filesystem, mở file, đọc, ghi, đồng bộ và đóng file cho các hệ thống nhúng [4].

EduFramework sử dụng FatFs bên dưới SD Card Device nhưng ẩn các cấu trúc nội bộ khỏi application. Sau khi `SD_Begin()` thành công, SD Card đã được khởi tạo và filesystem được mount. Application có thể tiếp tục mở file và ghi dữ liệu bằng các API của EduFramework [6].

Vòng đời file trong bài thực hành được giới hạn ở bốn bước chính:

`Initialize` → `Open` → `Write` → `Close`.

Việc đóng file sau khi ghi là một bước quan trọng. Tài liệu FatFs khuyến nghị file đang mở cần được đóng sau phiên truy cập; dữ liệu và thông tin filesystem còn được cache có thể chưa được cập nhật an toàn nếu nguồn bị ngắt hoặc thiết bị lưu trữ bị tháo khi file vẫn đang mở [4].

Ghi chú

Chỉ tháo SD Card sau khi chương trình đã hoàn thành việc ghi và đóng file. Trong bài thực hành, thông báo `Logging complete.` được in sau `SD_Close()` và `SD_End()` để xác nhận chu trình lưu trữ đã kết thúc.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài lab sử dụng MaaZEDU Development Board với vi điều khiển S32K144 thuộc họ S32K1xx [5]. Module MicroSD làm bộ nhớ ngoài, trong khi biến trở cung cấp tín hiệu analog thay đổi để tạo dữ liệu kiểm thử.

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
MicroSD Card Module	Module lưu trữ dùng để kết nối SD Card với giao diện SPI của MaaZEDU.
MicroSD Card	Thiết bị lưu dữ liệu của bài thực hành.
Potentiometer	Tạo điện áp analog thay đổi để sinh các mẫu ADC.
Jumper wires	Kết nối nguồn, SPI, chip-select và tín hiệu analog.
USB Cable	Kết nối board với máy tính để cấp nguồn, nạp chương trình và sử dụng Serial Monitor.

Bảng 17.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ chân sử dụng

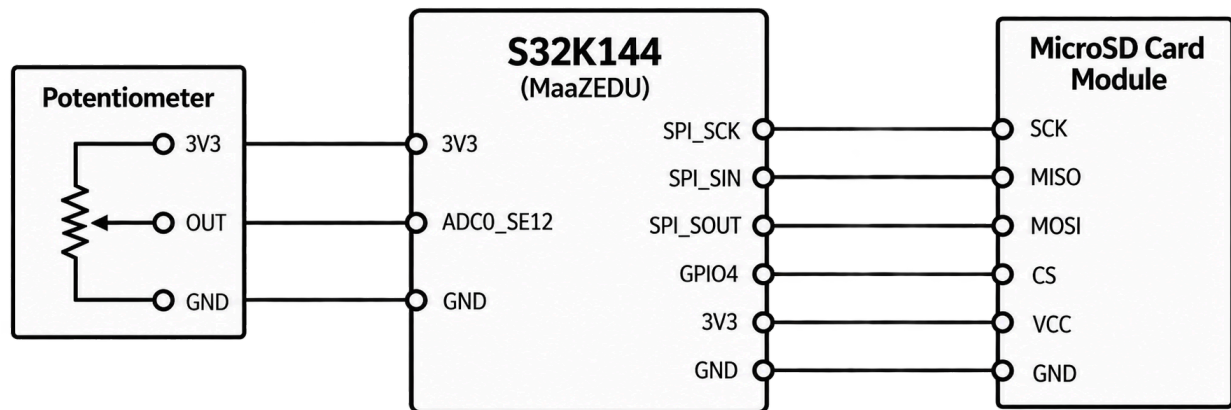
SD Card Device sử dụng các đường SPI của EduFramework cho clock và dữ liệu. Chân `GPIO4` được chọn làm software chip-select của module MicroSD. Biến trở được đọc qua `ADC0_SE12`. MaaZEDU Guide xác định các chân SPI và GPIO tương ứng, trong khi EduFramework cung cấp các Logical Pin sử dụng trong mã nguồn [7] [6].

Thành phần	Logical Pin	MCU Pin	Chức năng
MicroSD <code>SCK</code>	<code>SPI_SCK</code>	<code>PTB14</code>	SPI Clock
MicroSD <code>MOSI</code>	<code>SPI_SOUT</code>	<code>PTB16</code>	SPI Data: MaaZEDU → MicroSD
MicroSD <code>MISO</code>	<code>SPI_SIN</code>	<code>PTB15</code>	SPI Data: MicroSD → MaaZEDU
MicroSD <code>CS</code>	<code>GPIO4</code>	<code>PTD12</code>	Software Chip-Select
Potentiometer <code>OUT</code>	<code>ADC0_SE12</code>	<code>PTC14</code>	Analog Input

Bảng 17.3.2. Ánh xạ tín hiệu sử dụng trong Lab 17

3.3 Kết nối mạch

Module MicroSD được cấp nguồn từ `3V3` trong cấu hình đã kiểm chứng của bài lab. `SCK`, `MOSI`, `MISO` được nối tới các đường SPI tương ứng, còn `CS` được nối tới `GPIO4`. Biến trở có hai đầu ngoài nối với `3V3` và `GND`; chân tín hiệu `OUT` nối với `ADC0_SE12`.



Hình 17.3.1. Sơ đồ kết nối MicroSD và biến trở với MaaZEDU Development Board

Ghi chú

Không tháo SD Card trong khi chương trình đang ghi dữ liệu. Sau khi Serial Monitor hiển thị `Logging complete.`, file đã được đóng và SD Device đã kết thúc phiên hoạt động của bài thực hành.

4 EduFramework API

Bài thực hành chính sử dụng SD Card Device API để quản lý toàn bộ chu trình lưu trữ. Các chi tiết về SD command, block access và FatFs object không xuất hiện trong application. Application chỉ giữ một `SD_File_t` và gọi các API tương ứng với các bước khởi tạo, mở, ghi và đóng file [6].

4.1 SD_File_t

`SD_File_t` là file handle do SD Card Device quản lý. Cấu trúc này được truyền cho các API thao tác file sau khi `SD_Open()` thành công. Application không cần truy cập hoặc thay đổi trường `handle` bên trong trực tiếp [6]. Ví dụ:

```
SD_File_t file = {SD_INVALID_HANDLE};
```

4.2 SD_Begin()

SD_Begin

Khởi tạo SD Card qua SPI và mount filesystem sử dụng chân software chip-select đã chọn.

Cú pháp

```
SD_Begin(csPin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
csPin	Digital Logical Pin	Chân chip-select của SD Card. Bài lab sử dụng <code>GPIO4</code> .

Giá trị trả về

`true` nếu SD Card được khởi tạo và filesystem được mount thành công; `false` nếu quá trình khởi tạo hoặc mount thất bại.

Trong bài lab, chỉ cần một lệnh `SD_Begin(GPIO4)` để chuẩn bị cả giao tiếp SD và filesystem trước khi mở file [6] .

4.3 SD_Open() và SD_FILE_WRITE**SD_Open**

Mở một file và gán file handle cho biến `SD_File_t` .

Cú pháp

```
SD_Open(&file, path, mode);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
file	SD File	Địa chỉ của biến <code>SD_File_t</code> dùng để quản lý file đang mở.
path	File path	Tên hoặc đường dẫn file cần mở, ví dụ <code>"data.csv"</code> .
mode	File mode	Chế độ truy cập file. Bài lab sử dụng <code>SD_FILE_WRITE</code> .

Giá trị trả về

`true` nếu file được mở thành công; `false` nếu không thể mở hoặc tạo file.

`SD_FILE_WRITE` được ánh xạ tới chế độ tạo file để ghi. Nếu file cùng tên đã tồn tại, nội dung cũ được thay thế. Cách xử lý này tương ứng với cơ chế `FA_CREATE_ALWAYS` | `FA_WRITE` của FatFs [6] [4] .

4.4 SD_WriteLine()**SD_WriteLine**

Ghi một chuỗi ký tự vào file và thêm ký tự kết thúc dòng.

Cú pháp

```
SD_WriteLine(&file, text);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
file	SD File	File handle đã được mở bằng <code>SD_Open()</code> .
text	Text	Chuỗi ký tự cần ghi vào file.

Giá trị trả về

`true` nếu thao tác ghi hoàn thành; `false` nếu file không hợp lệ hoặc quá trình ghi thất bại.

Trong chương trình, mỗi mẫu ADC được chuyển thành một dòng text trước khi truyền cho `SD_WriteLine()` [6] .

4.5 SD_Close() và SD_End()

SD_Close() kết thúc phiên truy cập của một file đang mở. Sau khi đóng thành công, file handle không còn đại diện cho một file đang mở. Việc đóng file cũng là bước cần thiết để hoàn tất cập nhật dữ liệu của filesystem [4] [6] .

SD_Close

Đóng file đang được quản lý bởi SD_File_t .

Cú pháp

```
SD_Close(&file);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
file	SD File	File handle cần đóng.

Giá trị trả về

true nếu file được đóng thành công; false nếu file handle không hợp lệ hoặc thao tác đóng thất bại.

Sau khi file đã được đóng, SD_End() giải phóng các tài nguyên filesystem còn mở, unmount filesystem và kết thúc hoạt động của giao diện SD trong EduFramework [6] .

```
SD_Close(&file);  
SD_End();
```

5 Bài thực hành

5.1 Yêu cầu

Xây dựng một data logger sử dụng biến trở làm nguồn dữ liệu analog. Chương trình đọc ADC0_SE12 và lưu mười mẫu vào file data.csv trên SD Card. Hai lần lấy mẫu liên tiếp cách nhau 1000 ms .

Chương trình cần thực hiện theo trình tự:

Khởi tạo SD → Mở file → Thu 10 mẫu ADC → Ghi file → Đóng file .

Serial Monitor được sử dụng để hiển thị số thứ tự mẫu và giá trị ADC trong quá trình chạy. Sau khi hoàn tất, SD Card có thể được tháo và kiểm tra trên máy tính.

5.2 Chương trình

Trong src/main.c , triển khai chương trình đã được kiểm chứng trên phần cứng như sau:

```
#include "Arduino.h"
#include "sd_card.h"
#include <stdio.h>
int main(void)
{
    SD_File_t file = {SD_INVALID_HANDLE};
    int adcValue = 0;
    uint8_t sample = 0U;
    char line[32];
    setup();
    Serial1_begin(9600U);
    if (false == SD_Begin(GPI04))
    {
        Serial1_println("SD initialization failed.");
        while (1)
        {
        }
    }
    if (false == SD_Open(&file, "data.csv", SD_FILE_WRITE))
    {
        Serial1_println("File open failed.");
        while (1)
        {
        }
    }
    SD_Writeline(&file, "sample,adc_value");
    for (sample = 1U; sample <= 10U; sample++)
    {
        adcValue = analogRead(ADC0_SE12);
        snprintf(
            line,
            sizeof(line),
            "%u,%d",
            (unsigned int)sample,
            adcValue);
        SD_Writeline(&file, line);
        Serial1_print("Sample ");
        Serial1_printInt(sample);
        Serial1_print(" | ADC: ");
        Serial1_printlnInt(adcValue);
        delay(1000U);
    }
    SD_Close(&file);
    SD_End();
    Serial1_println("Logging complete.");
    while (1)
    {
    }
    return 0;
}
```

Mã nguồn 17.5.1. Chương trình ghi dữ liệu ADC vào SD Card

`setup()` khởi tạo các thành phần nền tảng của EduFramework và `Serial1_begin(9600U)` chuẩn bị Serial Monitor. `SD_Begin(GPI04)` sau đó khởi tạo SD Card và mount filesystem. Nếu khởi tạo thất bại, chương trình dừng tại vòng lặp lỗi để không tiếp tục thao tác với thiết bị lưu trữ chưa sẵn sàng [6].

`SD_Open()` tạo `data.csv` với `SD_FILE_WRITE`. Dòng đầu tiên được ghi bằng `SD_WriteLine()` để mô tả hai giá trị được lưu trong mỗi mẫu. Trong vòng `for`, `analogRead(ADC0_SE12)` lấy giá trị ADC thô của biến trở. `snprintf()` tạo một chuỗi gồm số thứ tự mẫu và giá trị ADC; chuỗi này được ghi xuống file bằng `SD_WriteLine()`.

Mỗi lần lấy mẫu cách nhau `1000 ms`. Sau mười mẫu, `SD_Close()` đóng file và `SD_End()` kết thúc phiên sử dụng SD Card. Chương trình chỉ in `Logging complete.` sau hai thao tác này, vì vậy thông báo được dùng làm mốc để biết quá trình ghi của bài thực hành đã kết thúc.

Lưu đồ dữ liệu của ứng dụng:

Potentiometer → `analogRead()` → ADC value → `SD_WriteLine()` → SD Card.

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board, sau đó mở Serial Monitor với baud rate `9600`.

Trong khoảng mười giây lấy mẫu, xoay biến trở để tạo các mức điện áp khác nhau tại `ADC0_SE12`.

Serial Monitor cần hiển thị mười mẫu và kết thúc bằng thông báo:

```
Sample 1 | ADC: 4095
Sample 2 | ADC: 4095
Sample 3 | ADC: 1958
...
Sample 10 | ADC: 2868
Logging complete.
```

Sau khi xuất hiện `Logging complete.`, tháo SD Card và mở `data.csv` trên máy tính. Một kết quả đã được kiểm chứng trên phần cứng có dạng:

```
sample,adc_value
1,4095
2,4095
3,1958
4,1287
5,748
6,0
7,0
8,766
9,1654
10,2868
```

Giá trị ADC cụ thể phụ thuộc vào vị trí biến trở. Mục tiêu kiểm chứng không phải đạt đúng các con số trên mà là file được tạo thành công, có đủ mười mẫu và giá trị thay đổi khi vị trí biến trở thay đổi.

KẾT QUẢ MONG ĐỢI

SD Card được khởi tạo thành công và file `data.csv` được tạo. Chương trình ghi đủ mười mẫu ADC với khoảng lấy mẫu `1000 ms`, sau đó đóng file và kết thúc SD Device. Dữ liệu trong file phải tương ứng với các mẫu đã hiển thị trên Serial Monitor và thay đổi theo vị trí của biến trở.

6 Mở rộng

Bài thực hành chính sử dụng `SD_FILE_WRITE` để mỗi lần chạy tạo một bộ dữ liệu mới. SD Card Device còn cung cấp các API để nối thêm dữ liệu, đọc file, đồng bộ dữ liệu và truy vấn thông tin lưu trữ [6].

API / Mode	Mục đích
<code>SD_FILE_APPEND</code>	Mở file để ghi tiếp vào cuối; file được tạo nếu chưa tồn tại.
<code>SD_Flush()</code>	Đồng bộ dữ liệu đang chờ của file xuống thiết bị lưu trữ mà không đóng file.
<code>SD_Read()</code>	Đọc một số byte từ file đang mở vào buffer của application.
<code>SD_Exists()</code>	Kiểm tra file hoặc thư mục theo đường dẫn có tồn tại hay không.
<code>SD_GetCapacity()</code>	Đọc tổng dung lượng và dung lượng trống của filesystem.

Bảng 17.6.1. Một số chức năng mở rộng của SD Card Device

Chế độ `SD_FILE_APPEND` khác với `SD_FILE_WRITE` : dữ liệu mới được ghi ở cuối file thay vì thay thế nội dung cũ. Trong EduFramework, hai mode này lần lượt được ánh xạ tới `FA_OPEN_APPEND` | `FA_WRITE` và `FA_CREATE_ALWAYS` | `FA_WRITE` của FatFs [4] [6] .

Ghi chú

`SD_Flush()` hữu ích khi một file cần được giữ mở trong thời gian dài nhưng ứng dụng muốn đồng bộ dữ liệu đã ghi xuống storage. Hàm này không thay thế việc `SD_Close()` khi phiên truy cập file thực sự kết thúc.

6.1 Bài tập mở rộng

Thay `SD_FILE_WRITE` bằng `SD_FILE_APPEND` và điều chỉnh chương trình để mỗi lần reset board, mười mẫu mới được thêm vào cuối file thay vì xóa dữ liệu của lần chạy trước. Sau ít nhất hai lần chạy, kiểm tra file trên máy tính và xác nhận dữ liệu của cả hai phiên vẫn được giữ lại.

Một hướng mở rộng khác là thay potentiometer bằng nguồn dữ liệu đã sử dụng ở các bài lab trước, chẳng hạn nhiệt độ từ NTC hoặc dữ liệu chuyển động từ MPU6050. Khi đó, cấu trúc lưu trữ của Lab 17 có thể được tái sử dụng mà không cần thay đổi cơ chế quản lý SD Card.

7 Tài liệu tham khảo

- [1] Youcef Bouchekioua, Hiroshi Matsui, Shigeru Watanabe, *A Versatile and Fast-Sampling Rate Wearable Analog Data Logger* , MethodsX, Vol. 10, 102098, 2023.
- [2] Patricia A. Beddows, Edward K. Mallon, *Cave Pearl Data Logger: A Flexible Arduino-Based Logging Platform for Long-Term Monitoring in Harsh Environments* , Sensors, Vol. 18, No. 2, 530, 2018.
- [3] SD Association, *Physical Layer Simplified Specification* , SD Specifications - Part 1 Simplified, Rev. 9.10, 2023.
- [4] ChaN, *FatFs - Generic FAT Filesystem Module* , FatFs Documentation.
- [5] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.
- [6] EduFramework, *SD Card Device API* , EduFramework v3.0.0 Source Code.
- [7] MaaZEDU, *MaaZEDU Development Board Guide* .